Investigate, Fix or Escalate

The Prototype and its Code: A Complete Guide

Opol Charis
BSc Computer Science, Uganda Christian University

Department of Computing and Technology

10 November 2026

This guide explains every part of the repository that the prototype depends on: the scenario model, the simulator, the safety mechanism, the learning agent and its baselines, the ticket-and-log readers, the triage loop, delegated checks, fix verification, the actions taken after a decision, the live hidden-fault demo, the dashboards, the command line and the tests. It describes the code as it stands on 10 November 2026 (pilot model v2, settings v3, agent `v3_shielded_seed0`). All data are pilot data from one developer and fictional examples, not Directorate data.

Contents

1 The prototype in one page **3**

2 Architecture **4**

3 Repository map **4**

4 The scenario model **6**

4.1 Causes . . . 6

4.2 Checks, symptoms and context . . . 6

4.3 Operational rules . . . 7

4.4 Settings (v3, fix with verification) . . . 7

5 The simulator: `TriageEnv` **7**

6 The safety mechanism **8**

7 The agents **8**

7.1 Masked DQN . . . 8

7.2 Baselines and the exact policy . . . 9

7.3 Training and evaluation . . . 9

8 The prototype pipeline in detail **9**

8.1 Readers: from text to a form . . . 9

8.2 The triage loop . . . 10

8.3 Delegated check D11 . . . 10

8.4 Verification . . . 11

8.5 Actions after the decision . . . 11

8.6 Live mode: hidden-fault incidents . . . 11

8.7 The local language model . . . 12

9 Using the prototype **12**

9.1 Set-up . . . 12

9.2 Command line . . . 12

9.3 Dashboards . . . 12

10 Tests **13**

11 Results so far **13**

12 Limitations to state honestly **14**

13 How to extend it **14**

14 Questions you are likely to be asked **14**

1 The prototype in one page

What it does. The prototype takes an IT incident as people actually report it, a helpdesk ticket in plain English plus an application log, and decides what to do with it. At each step it can

1. **investigate**: run one of 11 diagnostic checks (D01–D11),
2. **fix**: apply the remedy for one of 8 causes (C01–C08), or
3. **escalate**: hand the incident to a person.

The decision is made by a **deep Q-network (DQN)** trained entirely in simulation. Every proposal goes through an independent, rule-based **safety mechanism** first. A forbidden fix is sent back to the agent once, and a second forbidden proposal in a row escalates the incident. Every fix is then **verified**: a code change is tested on a staging copy of a repository, and other remedies are judged by whether the symptoms clear. When the decision is final the prototype **acts**. It opens a *draft* pull request, writes a fix order to the owner, or writes a routed escalation report. Nothing is merged and no message is really sent.

The division of labour, which is the most important design idea:

Component	Its only job
Reader (keywords or a language model)	Turns text into a fixed *form*: symptoms, context, check results, each with a verbatim quote. It never names a cause or an action.
DQN agent	Chooses the next action from the form. It never sees the text.
Safety mechanism	Checks each proposed fix against the rules. It is not learnt, and the agent cannot change it.
Verifier	Says whether a fix cleared the symptoms.
Actions layer	Carries out the final, re-checked decision: a draft PR, a fix order or an escalation.

One sentence for a viva: a language model may *read*, the trained agent *decides*, the rule checker *permits*, staging tests *verify*, and a person *merges*. No component does another's job.

Three ways in.

- **Command line:** `python-msrc.prototype.cli--ticket...--logs...` (Section 9).
- **Prototype dashboard** (Streamlit, port 8501): `dashboard/prototype_app.py`.
- **Live demo dashboard** (port 8502): `dashboard/live_app.py`, started by `run_live_demo.bat`. It shows incidents with hidden faults that nobody wrote by hand.

2 Architecture

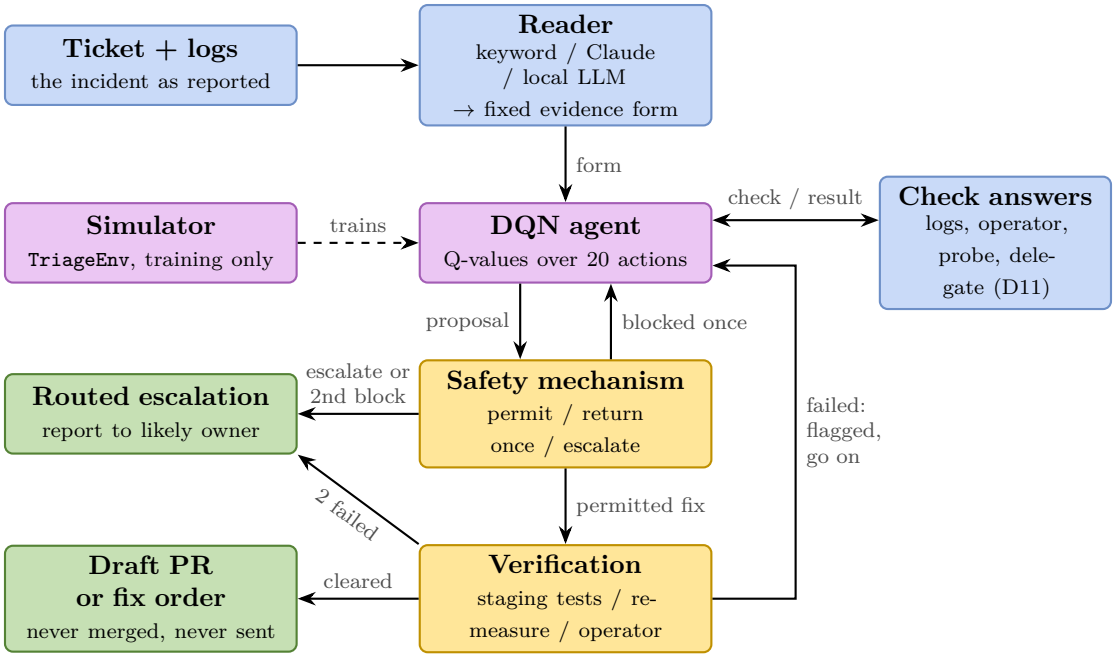


Figure 1: The prototype pipeline. Blue: input and evidence; pink: learning; yellow: rules and verification; green: what happens afterwards. The simulator is used only for training; at run time the same environment class is fed by the reader instead (`LiveTriageEnv`).

The key engineering decision is that the *same* environment, observation, action mask, safety mechanism and network are used in training, in evaluation and in the prototype. The prototype replaces only the source of evidence. In training a random incident is drawn from the scenario model. In the prototype the symptoms and context come from the reader, and a check's result is looked up only when the agent asks for it (`LiveTriageEnv` in `src/prototype/triage.py`).

3 Repository map

Path	What it is
Core study (simulation, rules, agents)	
`src/sim/load_model.py`	Reads an elicitation workbook (.xlsx) into the scenario-model JSON. Provenance (`--rater`) is passed in, so no name reaches the JSON.
`src/sim/scenario.py`	`Scenario` dataclass and `build_scenario`: turns model + settings into arrays (likelihoods, prior, costs, rules, verification).
`src/sim/triage_env.py`	`TriageEnv`, the Gymnasium environment: one episode is one incident; also `Layout` to decode observations.
`src/rules/safety.py`	`RuleChecker`, `SafetyMechanism` (enforces), `RuleMonitor` (records only, optional penalty).
`src/agents/dqn.py`	Masked DQN (Stable-Baselines3 subclass), wrappers, hyperparameters, `AgentPolicy`.
`src/agents/baselines.py`	Always escalate, initial-symptom, scripted checklist, threshold policies.

Path	What it is
`src/agents/belief.py`	Bayesian posterior over causes and expected information gain.
`src/agents/exact.py`	Exact optimal policy by dynamic programming (v1 settings only).
`src/agents/monitoring.py`	Training callback: scores the greedy agent every N steps, writes `metrics.jsonl`, saves checkpoints.
`src/evaluation.py`	Runs a policy on fixed seeds and computes every Chapter 3 measure.
Prototype	
`src/prototype/reading.py`	`Finding`, `Reading`, the offline `KeywordReader`.
`src/prototype/llm.py`	`ClaudeReader`: schema-constrained form, redaction, quote check.
`src/prototype/local_llm.py`	Free local model via Ollama: `LocalReader`, `HybridReader`, `LocalDelegate`.
`src/prototype/delegate.py`	Delegated check D11: `ClaudeDelegate`, `PatternDelegate`.
`src/prototype/triage.py`	`LiveTriageEnv` and `triage()`, the decision loop and its report.
`src/prototype/verify.py`	Fix verification: staging tests, operator answer, demo case answer.
`src/prototype/actions.py`	Routing, handover reports, draft pull requests, simulated outbox.
`src/prototype/live.py`	Live hidden-fault incidents: draws, writes ticket and log, probes, re-measures, scores.
`src/prototype/cli.py`	Command line.
Data, configuration and examples	
`experiments/pilot/pilot_model_v2.json`	The scenario model the prototype uses (8 causes, 11 checks, 4 symptoms, 2 context attributes, 5 rules).
`experiments/pilot/pilot_settings_v3.json`	Researcher settings: rewards, check minutes, priors, noise, verification.
`experiments/pilot/prototype_config.json`	Which model, settings and agent run to use; reader keywords, log patterns and thresholds; context defaults; action settings.
`prototype/examples/`	Eight fictional cases (ticket, logs, optional `verification.json`) and `cases.json` with the intended cause and expected reading.
`prototype/toy_repo/`	Fictional student portal with four planted bugs (one per code-fault cause) and its tests.
`prototype/toy_repo_fixes/`	The pre-written fix and explanation for each planted bug.
`results/pilot/runs/`	One folder per training run: `config.json`, `metrics.jsonl`, `final.zip`.
`results/pilot/outbox/`	Simulated messages written by the actions layer.
Interfaces and tests	
`dashboard/app.py`	Training dashboard: training curves, the agent's "thinking", early prototype page.
`dashboard/prototype_app.py`	Full prototype dashboard (seven tabs).
`dashboard/live_app.py`	Live demo dashboard (Live incident and Stress test only).

Path	What it is
`run_live_demo.bat`	Starts the live demo on port 8502.
`tests/`	212 tests (Section 10).

4 The scenario model

Everything the agent knows about the world comes from the *scenario model*, a JSON file read from an elicitation workbook (`src/sim/load_model.py`). The pilot model v2 was rated by one software developer with three years' experience. It is not Directorate data.

4.1 Causes

ID	Category	Cause	Freq.	Code fault	Owner
C01	Infrastructure	Slow external service, outbound call has no timeout	Occasional	yes	Integrations dev.
C02	Database	Long-running transaction holds row locks	Common	no	DBA
C03	Database	Missing index on a growing table	Rare	no	DBA
C04	Application	Repeated queries per request (N+1)	Common	yes	Backend lead
C05	Infrastructure	Cache failing silently, every request recomputes	Rare	yes	Backend lead
C06	Application	Bad release or migration breaks a code path	Occasional	no	Release manager
C07	Database	Duplicate rows where one was expected	Common	no	DBA
C08	Authentication	Sign-in writes collide on the user row	Common	yes	Identity lead

Remedies: C01 set a timeout; C02 kill the blocking transaction; C03 add the index; C04 batch the queries; C05 fix the cache serialisation; C06 roll back the release; C07 remove the duplicates; C08 move the token write off the sign-in path. "Code fault" causes are fixed by a code change, so the prototype can open a draft pull request for them.

4.2 Checks, symptoms and context

ID	Check	Effort	ID	Check	Effort
D01	Long outbound span in traces	Low	D07	Query count in a slow trace	Medium
D02	Lock-wait errors in logs	Low	D08	Cache errors in logs	Low
D03	DB connection pool usage	Low	D09	Recent release or migration	Low
D04	API node CPU	Low	D10	Failure scope	Medium
D05	Long transaction in the DB	Medium	D11	Full log analysis (*delegated*)	High
D06	Query plan of slowest query	High			

Symptoms: S01 server error on a page; S02 pages slow or timing out; S03 users can't sign in or get logged out; S04 a save or approval fails after a long wait. **Context:** X01 "affected system is production" (Yes/No); X02 priority (Low/Medium/High).

The model also holds two likelihood matrices, rated on a verbal scale and converted to probabilities: $\theta_{ij} = P(\text{check } i \text{ positive} \mid \text{cause } j)$ and $P(\text{symptom } i \text{ present} \mid \text{cause } j)$.

4.3 Operational rules

Each rule forbids one cause's remedy in a context *unless* a named check has been observed:

Rule	Forbids fix for	When	Unless observed / reason
R01	C02 kill transaction	production	D05. Killing the wrong transaction rolls back valid work.
R02	C06 roll back	production	D09. A rollback restarts every node; migrations may not reverse.
R03	C07 remove duplicates	production	D10. Other records may depend on the rows; deletes are hard to undo.
R04	C03 add index	priority High	D06. Index builds on large tables can block live traffic.
R05	C01 set timeout	production	D01. A wrong timeout can make failures look like real results.

4.4 Settings (v3, fix with verification)

Settings are set by the researcher, not elicited (`pilot_settings_v3.json`). Rewards are in hours (unit 60 minutes):

- check cost: Low 5, Medium 15, High 30 minutes;
- escalation $c_{\text{esc}} = 120$ min $= 2$ h (elicited); correct fix reward $+5$;
- prior from frequency weights Rare 1, Occasional 2, Common 4;
- context: production 80%; priority Low 30%, Medium 50%, High 20%;
- $p_{\text{noise}} = 0.1$ (a check result is reversed), $p_{\text{miss}} = 0$, step limit 15;
- verification: each remedy has an apply time and a harm weight. Mean harm of a wrong fix $=$ (wrong_fix_multiple $- 1$) $\times\, c_{\text{esc}} = 4 \times 2 = 8$ h, spread by harm weights (C07 2.0, C02 and C06 1.5, C03 1.0, C01 0.75, code faults 0.5). After **2 failed fixes** a person takes over.

`build_scenario` validates all of this: rewards must satisfy correct $>$ $-$escalate $>$ $-$wrong, every frequency and effort must have a value, context probabilities must sum to one, and with verification a correct fix must beat escalation for every remedy. An optional *overlap* ω blends the likelihoods of causes in the same category: $\theta'_{ij} = (1-\omega)\theta_{ij} + \omega\,\overline{\theta_{i\cdot}}$.

5 The simulator: `TriageEnv`

One episode is one incident. At `reset(seed)` everything is drawn once, so the same seed gives every policy the same incident:

1. the hidden cause $j \sim$ prior; the context from its probabilities;
2. the symptoms, each present with probability `symptom_theta[i,j]`;
3. each check's true result $\sim \theta_{ij}$, then reversed with probability p_{noise};
4. whether each check is available $(1 - p_{\text{miss}})$.

Actions (20 with model v2): 11 checks, 8 fixes and 1 escalation, in that order.

Observation (70 numbers, all in $[0, 1]$):

- each check's status one-hot over {not requested, unavailable, negative, positive} (4×11);
- each symptom one-hot over {present, absent, not reported} (3×4);

- context: one number for a two-valued attribute, one-hot otherwise $(1 + 3)$;
- step count / max steps and a "last proposal was blocked" flag (2);
- with verification: one "this fix was tried and failed" flag per cause (8).

During training the 20-value action mask is appended, so the network sees 90 inputs.

Action mask: a check can be run only once; a failed fix cannot be repeated; a blocked action is masked for the next step.

Rewards and endings (v3):

Action	Effect
Run check i	$-\text{cost}_i$; the status becomes positive / negative / unavailable
Fix the true cause	$+5-$ apply cost; episode ends *correct*
Fix a wrong cause	$-$(apply cost + harm); that fix is flagged and masked; after 2 failed fixes, also -2 and the episode ends *fix_limit*
Escalate	-2; ends *escalated*
Step limit (15)	-2; ends *step_limit* (counted as an escalation)

Without verification (v1/v2) any fix ends the episode, correct (+5) or wrong (−10).

Reference label (evaluation only, never shown to the agent): an incident "should be escalated" if a rule would block the true cause's fix with no lifting check available, or if even the full evidence points to a different cause. This label drives the escalation precision, recall and MCC.

6 The safety mechanism

`src/rules/safety.py` is deliberately separate from both the agent and the environment.

- `RuleChecker.forbidding(action,context,observed)` returns the rules that forbid an action now. Only fixes can be forbidden; checks and escalation are always allowed. A rule is lifted once its `unless_observed` check has returned a result ("unavailable" does not count).
- `SafetyMechanism` (the *shielded* agent): a permitted proposal passes unchanged. The first refusal uses a step with no reward, sets the blocked flag and masks that action. A second refusal in a row calls `force_escalation()`. **No prohibited action ever reaches the environment.**
- `RuleMonitor`: lets everything through and only counts violations. With `penalty=10` it subtracts 10 per violation, which gives the *penalty-only* agent. With no penalty it is the *unconstrained* agent.

In the prototype the rule checker is used a *second* time on the final decision with the final evidence (`decision["permitted"]`), and `actions.py` refuses to act unless this is `True`.

7 The agents

7.1 Masked DQN

`src/agents/dqn.py` subclasses Stable-Baselines3's DQN:

- `MaskInObservation` appends the action mask to the observation, and `MaskedQNetwork` sets masked Q-values to -10^9. Masking therefore covers both action choice and the max in the learning target.

- **Masked exploration with a check bias:** random actions respect the mask, and 80% of them are checks, so exploration reaches long investigations.
- Optional **Double DQN** target (online network picks, target network values), which the v3 runs use.
- Rewards are divided by 10 during training only (`ScaleReward`); `q_values()` returns them unscaled, in hours.

Hyperparameters of the prototype's agent (`results/pilot/runs/v3_shielded_seed0`): 500,000 steps, network 2×128, learning rate 10^{-4}, buffer 200,000, batch 64, $\gamma = 1$, target update every 1,000 steps, ε from 1.0 to 0.05 over the first 30%, Double DQN on. Seed 0 of three was chosen on the training monitor's return, not on evaluation incidents.

7.2 Baselines and the exact policy

- **Always escalate.**
- **Initial-symptom:** fixes the most probable cause given the symptoms, with no checks.
- **Scripted (k):** runs a fixed list of k checks ordered by average information gain, then fixes the most probable cause.
- **Threshold (τ):** runs the most informative check until a permitted cause reaches probability τ, then fixes it; escalates when the checks run out.

The baselines are given the likelihood model (Bayesian posterior, `belief.py`); the DQN is not. `exact.py` solves the v1 problem exactly by dynamic programming over all $3^{10} \times 16 \times 6$ evidence states, giving the optimal value. It refuses missing checks and verification.

7.3 Training and evaluation

`experiments/pilot/train_dqn.py` and `run_comparison.py` train the agents. `TrainingMonitor` scores the greedy agent on 1,000 fixed incidents every 25,000 steps. Seed ranges never overlap: training below 10^6, evaluation from 10^6, tuning from 2×10^6, monitoring from 3×10^6. `summarise()` computes return (with bootstrap CI), resolution accuracy, wrong-fix rates, checks and check cost, escalation rate/precision/recall/F1/MCC, proposed and executed violation rates, forced escalations and compliant success.

8 The prototype pipeline in detail

8.1 Readers: from text to a form

A reader returns a `Reading`: category, one-line summary, and a `Finding` for every symptom, context attribute and check. Each finding holds its `value`, the `evidence` (the exact sentence or log line) and its `source` (ticket, logs, default, operator, probe, delegate).

Reader	How it works
`keyword`	Offline. Keyword lists for the ticket; regular expressions with thresholds for the log (e.g. pool usage above 80%), time windows (e.g. a release within 24 h of the newest line) and median/max aggregation. All patterns live in `prototype_config.json`.
`claude`	Claude fills a JSON form whose schema is built from the scenario model, so a new model needs no code change. The prompt forbids diagnosis.
`local`	`HybridReader`: a local model (Ollama, `qwen3:4b`) reads only the ticket; the keyword patterns read the log, where numbers against thresholds matter. Takes about 30–45 s.
`local-full`	`LocalReader` on both ticket and log, slower. It asks for a compact list form with one worked example, then converts it back to the full form.

Safeguards on every language-model reader: (1) *redaction* before sending: e-mails, secrets, IPs, Ugandan phone numbers and long tokens become placeholders; (2) *schema-constrained output*, so the model cannot propose a fix; (3) the *quote check*: a finding whose quote is not in the (redacted) ticket or log is discarded with a warning, so an invented line cannot become evidence; (4) ticket text is declared to be data, not instructions.

Defaults: unstated context takes the stricter value (production, High priority), since both switch rules on. An unmentioned symptom counts as absent, as in training.

8.2 The triage loop

`triage(reading, scenario, model, agent, ask, delegate, ticket, logs, verify)`:

1. builds the symptom and context vectors from the reading, and resets the live environment wrapped in the safety mechanism, `SafetyMechanism(LiveTriageEnv(...))`;
2. at each step computes the agent's Q-values and takes the greedy action;
3. a **check** is answered lazily by `lookup`. A delegated check (D11) goes to the delegate. Otherwise the reading's finding is used. If the logs did not show it, `ask(check)` is called: the operator on the command line, or a `Probe` in live mode. With no answer the check is *unavailable*;
4. a **fix** goes through the safety mechanism, then the verifier. "Cleared" means resolved; "not cleared" flags the fix as failed and the agent continues; no answer stops the incident as *awaiting verification*;
5. each step records the action, its Q-value, the margin over the runner-up, the top-3 alternatives, the escalation value, the result and its evidence, any blocking rule, any verification, and the Bayesian belief over causes. The belief is *for the viewer only*; the agent never sees it.

The report holds the reading, the steps and the decision (outcome, checks run, minutes, blocked proposals, attempts, cause, remedy, `permitted`, `verified`). It also gives the three most probable causes, from the ratings, for whoever takes the incident next.

8.3 Delegated check D11

D11 "full log analysis" is an ordinary check marked `"delegated":true` with High cost. When the agent picks it, a fixed prompt template, filled from the check's definition (the agent does not write the prompt), is sent to a delegate (`claude`, `local`, or the offline `pattern`). The answer is reduced to positive, negative or not shown plus a quote. Only that result reaches the agent; the delegate's written report is kept for the person. The same redaction and quote check apply. In the pilot D11 asks whether the stack traces point into the application's own code (`portal/...`).

8.4 Verification

`src/prototype/verify.py` chains verifiers, and the first definite answer wins:

1. `StagingVerifier` (code faults only): copies the toy repository to a temporary folder, applies the patch, and runs the tests. The patch must break no test that passed before, *and* the incident's own reproduction tests (from `verification.json`) must now pass. "The patch passes some test" is not enough, because all planted bugs are present at once.
2. `AnswerVerifier`: the operator says whether the symptoms cleared.
3. `CaseVerifier` (demo only, `--demo-verify`): the cause the example case was written to show.

A verifier acts on nothing real. The patch that was verified is stored and is exactly the patch the later pull request carries (`StoredPatcher`).

8.5 Actions after the decision

`src/prototype/actions.py` is a demo layer and is not part of the evaluated study.

- **Routing:** a fix goes to the cause's owner. An escalation goes to the owner of the most probable cause if it reaches 50%, otherwise to the fallback (senior developer).
- **Code fault → draft pull request** on the toy repository: local (a branch in `results/pilot/demo_repo`) or GitHub (one configured throwaway repository). The patch is pre-written or written by Claude, and the tests are re-run.
- **Other fix → fix order**; **escalation → handover report** with the ticket, every step and its evidence, the most probable causes and the fixes already tried.
- Messages are written to `results/pilot/outbox/`. Nothing is sent.

Guarantees enforced in code: (1) nothing acts unless `permitted` is true; (2) PRs only for `code_fault` causes, and only as drafts; (3) a patch may only change an existing non-test source file; (4) a command allow-list (`gitinit/add/commit/checkout/clone/push/config`, `ghprcreate--draft`); pushing `main`/`master` or forcing is refused. Nothing ever merges.

8.6 Live mode: hidden-fault incidents

`src/prototype/live.py` creates incidents that nobody wrote and nobody has seen:

- `draw()` samples an incident exactly as the training environment does (cause, context, symptoms, check results with reversals). Three origins: *random*, *chosen* (someone plants the cause), or *your ticket* (the reader's symptoms, context and stated results fix the hidden system, and the cause is drawn given them).
- `write_ticket()` and `write_logs()` turn it into text: a ticket from phrase banks (never naming the cause), and a log with background traffic, failing requests, evidence lines for about 55% of checks (often *near misses* below a threshold or outside a window), and stack traces for D11.
- `Probe` answers a check the log does not cover from the hidden system, with a realistic console command and its output.
- `Remeasure` verifies non-code fixes by "re-measuring" the symptoms: they clear only if the fix was for the hidden cause. Code faults go through the staging tests against the hidden fault's reproduction tests.

- At the reveal: the hidden cause, the outcome, which results were reversed by noise, the reader's mistakes, and `hindsight()`, the cause that *all* check results together point to. When that is wrong, escalating was the sound choice.

8.7 The local language model

`local_llm.py` talks to Ollama at `http://localhost:11434` (model `qwen3:4b`, temperature 0, JSON-schema output), so nothing leaves the laptop. Environment variables `TRIAGE_LLM_URL`, `TRIAGE_LLM_MODEL`, `TRIAGE_LLM_KEY` switch it to any OpenAI-compatible endpoint. Use that only for fictional tickets, because the text then leaves the machine.

9 Using the prototype

9.1 Set-up

```
python -m venv --system-site-packages .venv
.venv\Scripts\python -m pip install -r requirements.txt
winget install Ollama.Ollama        (optional, for the local reader)
ollama pull qwen3:4b
```

The Claude reader needs `ANTHROPIC_API_KEY`. Use it only on fictional tickets unless the Directorate approves.

9.2 Command line

```
python -m src.prototype.cli --ticket prototype\examples\sign_in\ticket.txt
       --logs prototype\examples\sign_in\logs.txt --act
```

Flag	Meaning
`--readerkeyword|local|local-full|claude`	Who reads the text (default keyword, offline).
`--ask`	Ask you for checks the logs do not show, and whether a fix cleared the symptoms.
`--delegatepattern|local|claude|off`	Who answers D11 (default pattern).
`--act`	Carry the decision out (draft PR, fix order or escalation report).
`--prlocal|github|off`, `--patcherpre-written|claude`	Where PRs go and who writes the patch.
`--demo-verify`	Example cases only: verify against the intended cause.
`--runDIR`, `--jsonFILE`, `--incidentID`	Agent run, save the report, name the incident.

9.3 Dashboards

Prototype dashboard (`streamlitrundashboard/prototype_app.py`, port 8501), tabs: *Live incident*, *Stress test*, *Triage a ticket* (reading → steps → decision → verify → action, with the PR diff), *All example cases*, *Outbox and draft PRs*, and reader *accuracy*. The sidebar chooses the agent run, reader, delegate, actions, and how fixes are verified, and shows a green dot when the local model is ready.

Live demo (`run_live_demo.bat`, port 8502): only the Live incident and Stress test tabs. The run sheet is `documentation/live_demo.md`:

1. *Random hidden fault*: new incident (padlock closed) → let the agent work (an animated console with the belief bars, blocks and verifications) → reveal (padlock opened).
2. *Someone plants the fault*: an audience member picks the cause.
3. *Your own ticket*: someone dictates a ticket; the local model reads it.
4. *Stress test*: 20–500 fresh incidents in seconds (keyword reader, re-measuring), showing resolved %, first fix worked %, escalations, and how many escalations were sound (the evidence itself pointed to the wrong cause).

Training dashboard (`dashboard/app.py`): training curves of any run, and a "thinking" view that compares the agent's Q-values with the exact solution.

10 Tests

Run with `python-mpytest` (about five minutes). The table counts test functions; with parametrisation pytest collects 212 tests.

File	Functions	What it protects
`test_load_model.py`	12	Workbook reading, rule conditions, likelihood grids
`test_triage_env.py`	25	Drawing, observations, masks, rewards, verification, step limit
`test_safety.py`	26	Block-then-escalate, rule lifting, monitor and penalty
`test_baselines.py`	21	Posterior, information gain, each baseline, exact solver
`test_dqn.py`	10	Masking in network and exploration, Double DQN target
`test_prototype.py`	21	Readers, redaction, quote check, triage loop on the examples
`test_extension.py`	20	Delegation, routing, draft PR guarantees, command allow-list
`test_verify.py`	12	Verification in the environment
`test_prototype_verify.py`	8	Staging verifier and verifier chain
`test_live.py`	11	Live draws reproducible; ticket and log never name the cause; probing; fixes verified only for the hidden cause; never a wrong fix; own-ticket mode; local reader and delegate forms
Total	166	212 tests once parametrised; all pass

11 Results so far

Simulation (5,000 identical evaluation incidents, escalation 120 min, agents mean of 3 seeds; returns in hours):

Policy	Return	Resolved	Any wrong fix	Escalated	Checks	Rule broken
Always escalate	−2.00	0.0%	0.0%	100.0%	0.00	0.0%
Initial-symptom	−2.33	41.3%	28.6%	58.7%	0.00	0.0%
Scripted ($k = 9$)	−1.38	88.3%	29.7%	11.7%	9.00	0.0%
Threshold ($\tau = 0.65$)	−0.38	74.1%	21.4%	25.9%	4.72	0.0%
DQN + safety mechanism	−0.41	67.6%	22.5%	32.4%	5.73	0.0%
DQN unconstrained	−0.28	74.7%	24.9%	25.3%	6.05	7.6%
DQN penalty-only	−0.24	74.7%	25.4%	25.3%	4.98	0.0%

Behind the mechanism, 0 of 15,000 agent incidents executed a rule violation. Proposals that break a rule settle at 20–25%, because a block costs nothing in the pilot. Escalation rises and wrong fixes fall as the cost of escalation rises (60 / 120 / 240 min).

Prototype (eight fictional cases, keyword reader, agent v3): reading 100% correct on symptoms, context and checks. Six of eight cases end as intended: results_approval, sign_in, transcript_duplicates, class_results_slow and profile_cache are fixed and verified, and vague_slow (written to have no cause) has a wrong guess that fails verification and is then escalated. fees_release (C06) and payment_gateway (C01) are escalated instead of fixed. These cases show that the pipeline works; they do not measure accuracy.

12 Limitations to state honestly

- One rater (a developer), not the Directorate; several values are set by the researcher, including all verification settings.
- Three seeds per agent (ten planned); only the base ambiguity level was run.
- A blocked proposal costs nothing, so the shielded agent leans on the mechanism.
- Verification is assumed reliable. In live mode it is perfect by construction.
- The keyword reader is tuned to the example log format; the live log is written in that same format. Live incidents come from the agent's own world (the scenario model), so they show that the pipeline works, not that the model fits real incidents.
- The toy repository, the outbox and the PRs are fictional; nothing touches a real system.

13 How to extend it

- **Directorate model:** fill the Directorate workbook, then run `python -m src.sim.load_model WORKBOOK.xlsx model.json --label ... --rater "role"`. Add owners, `code_fault` and delegated flags. Point `prototype_config.json` at it, add keyword and pattern entries for each new symptom, context value and check, add verification settings for each remedy, and retrain. The Claude and local readers need no code change, because their form is built from the model.
- **A new check:** add it to the model (with likelihoods), give it a pattern in `checks` of the config (`covers`, `positive`, optional `above`, `within_hours`, `aggregate`), and for live mode an evidence line in `_evidence_lines` and a probe command.
- **A new code-fault fix:** a bug in `prototype/toy_repo`, a failing test, the fix in `prototype/toy_repo_fixes/<cause>/`, and a case with `verification.json` naming the reproduction test.

14 Questions you are likely to be asked

Does the language model make the decision?
No. It only fills a fixed form, and its schema has no place for a cause or an action. The trained DQN decides.

What stops a hallucinated log line from becoming evidence?
The quote check discards any finding whose quote is not literally in the ticket or log.

Can the agent break a rule?
Not behind the mechanism: a forbidden fix never reaches the environment, and the final decision is checked again before anything acts.

Why does the shielded agent still propose forbidden fixes?
A block is free in the pilot. Charging for it is the planned ablation.

Why can't a wrong fix "succeed"?

With verification the symptoms clear only for the right cause. A wrong fix is flagged, and the agent goes on or escalates after two.

Why did it escalate a case it could have fixed?

The value of escalating was higher given the evidence and the cost of a wrong, hard-to-undo remedy. The stress test also shows how many escalations were sound because the evidence itself pointed elsewhere.

Is any real data sent anywhere?

The keyword reader and Ollama run locally. The Claude reader redacts first and is for fictional tickets only. Messages go to a local outbox.